CE-1103 · Lección 01

Programación Orientada a Objetos

Modelar el mundo real con clases y objetos

Televisor.cs

```
class Televisor
{
    public bool encendido;
    public void Encender() {}
}
```

Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

// introducción

¿Qué aprenderemos hoy?

1. Distinguir tipos simples, compuestos, de valor y de referencia
2. Explicar qué es un tipo de dato abstracto (TDA) y su relación con las clases
3. Ubicar la POO entre los paradigmas de programación
4. Diferenciar clase y objeto: atributos y comportamiento
5. Aplicar los cuatro principios: abstracción, encapsulamiento, herencia y polimorfismo
6. Reconocer el polimorfismo en tiempo de ejecución y en tiempo de compilación

// conceptos esenciales

Tipos de datos

Define qué valores puede tomar una variable y qué operaciones admite. En Python:

int

Enteros: números sin decimales

1, 20, -5

float

Flotantes: números con decimales

3.14

str

Cadenas: secuencias de caracteres

"Hola, mundo!"

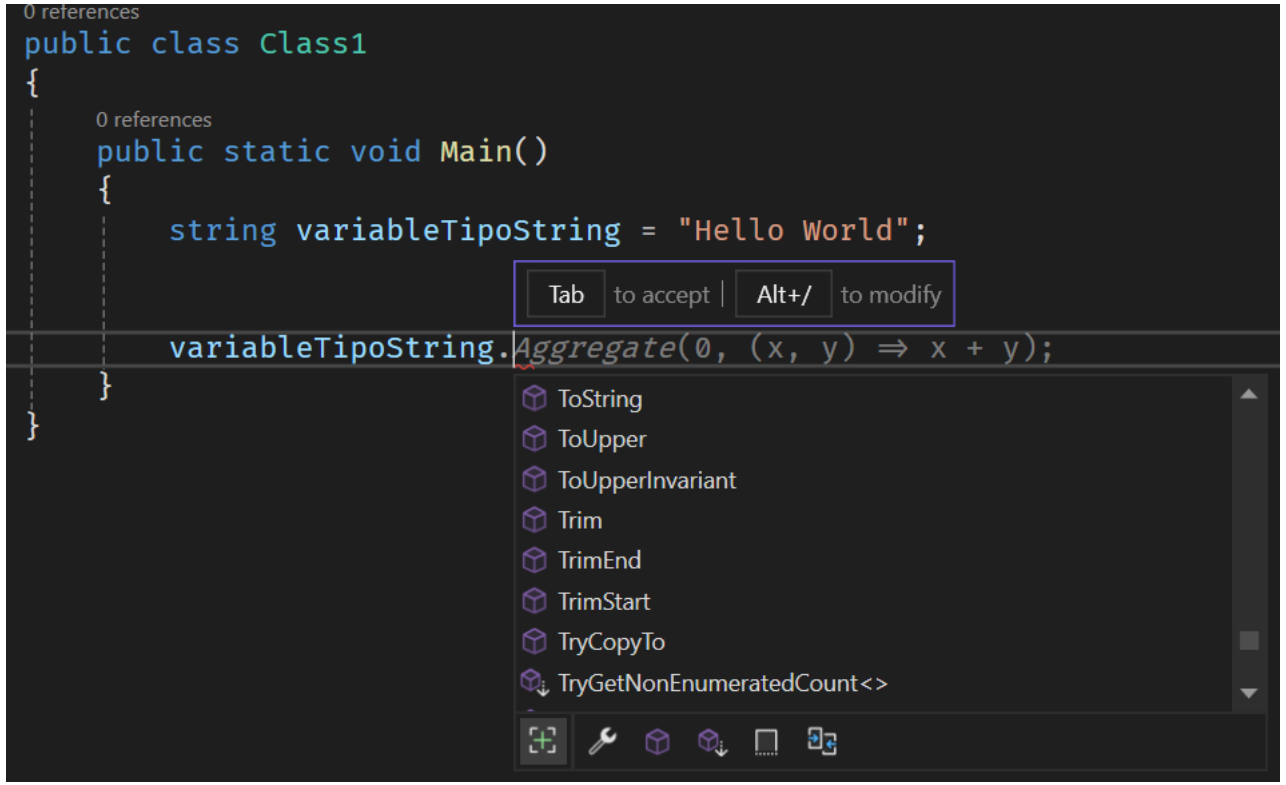
bool

Booleanos: valores de verdad

True, False

Operaciones: aritméticas con enteros y flotantes, concatenar cadenas y operaciones lógicas con booleanos.

Los IDE aprovechan los tipos de datos



Autocompletado según el tipo de dato: la variable es un `string`, así que se sugieren sus operaciones.

// conceptos esenciales

Tipo de dato simple vs. compuesto

Simple

- Representa **un único valor**
- Son los tipos básicos para valores individuales
- No tiene sentido prácticos separarlos en partes más pequeñas

vs

Compuesto

Representa una **colección de valores** de otros tipos de datos.

nombre **edad** **dirección**

Ejemplo: un tipo `Cliente`.

Los objetos se consideran tipos de datos compuestos.

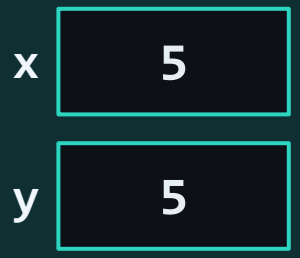
// conceptos esenciales

Tipo valor vs. tipo referencia

Tipo valor

Almacena el **valor** directamente en la memoria.

```
int x = 5;
int y = x;
```

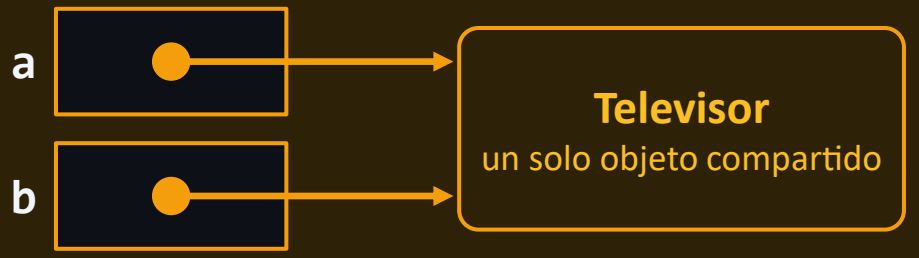


Dos copias independientes: cambiar y no afecta a x.

Tipo referencia

Almacena una **referencia** a una ubicación en memoria.

```
Televisor a = new Televisor();
Televisor b = a;
```



Usualmente los tipos **simples** son de **valor** y los **compuestos** de **referencia**. Los **objetos** son tipos de referencia.

// conceptos esenciales

Tipo de dato abstracto (TDA)

1

Modelo conceptual

Define un **conjunto de valores** y un **conjunto de operaciones** sobre ellos.

2

Solo la interfaz

Especifica **qué** operaciones existen, **no cómo** se implementan.

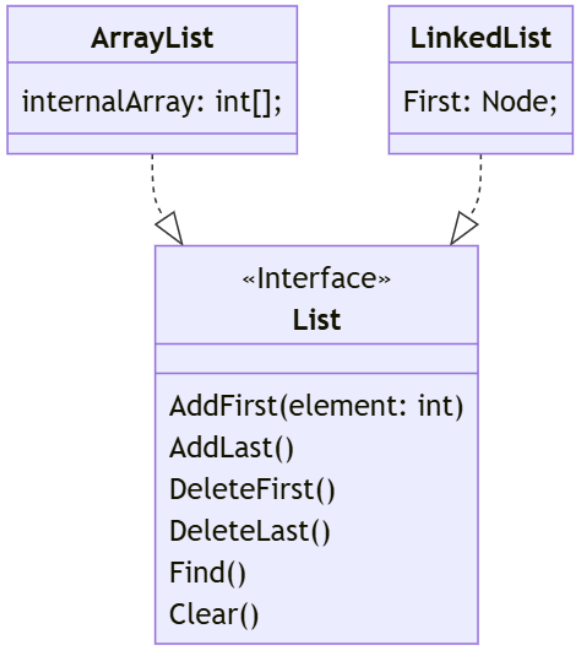
3

Dos vistas

Vista **lógica** (la interfaz) y vista **física** (la implementación).

Un TDA permite usar los datos **sin preocuparse** por los detalles de implementación.

Un TDA, varias implementaciones



Una interfaz, dos implementaciones

- Interfaz `List`: `AddFirst`, `AddLast`, `DeleteFirst`, `DeleteLast`, `Find` y `Clear`
- `ArrayList`: un arreglo (`int[]`) con **memoria estática**
- `LinkedList`: **Nodos** con **memoria dinámica**
- El API de `List` no debe revelar cómo se implementa por dentro

Quien usa `List` no necesita saber si por dentro hay **Nodos** o un **arreglo**.

// conceptos esenciales

De los TDA a las clases

qué

TDA

La **interfaz**: las operaciones que se pueden realizar con los datos.

+

cómo

Implementación

La **estructura de datos** que almacena y opera los datos.

=

POO

Clase

En programación orientada a objetos, **TDA + implementación** forman una clase.

Algunos lenguajes permiten definir **interfaces**: un TDA **puro**, sin implementación.

¿Qué es un paradigma?

Paradigma: el estilo fundamental de programar; cómo un lenguaje «ve» el mundo.

Funcional

El mundo como **matemáticas** y **funciones**.

Scheme, Lisp, F#

Imperativo

El mundo como **pasos**, **comandos** y estructuras.

C, Pascal

Lógico

El mundo como **axiomas**, **reglas de inferencia** y consultas.

Prolog

Objetos

El mundo como **objetos** con atributos y comportamiento.

Java, C#, Smalltalk

Ciudadano de primera clase: entidad que admite todas las operaciones disponibles para las demás (p. ej. las variables): asignarse, pasarse como argumento y devolverse.

¿Qué es la programación orientada a objetos?

1

Paradigma

Modela conceptos del **mundo real** como **objetos**.

2

Objetos

Tienen **atributos** (datos) y **comportamientos** (métodos).

3

Primera clase

Se pueden manipular y **pasar como argumentos** a funciones.

Construir software orientado a objetos: **definir clases** y crear **instancias** (objetos) que interactúan entre sí.

Todo objeto tiene atributos y comportamiento

Atributos

Lo que el objeto **es** o **tiene**:

- **color: negro**
- **marca: Lamborghini**
- **material: metal**
- **peso: alto**

*En una clase: los **atributos** (datos).*

vs

Comportamiento

Lo que el objeto **puede hacer**:

- **acelerar**
- **frenar**
- **girar**
- **reproducir música**

*En una clase: los **métodos** (funciones).*

Describir solo los atributos es incompleto: un objeto se conoce por sus **atributos** y por su **comportamiento**.

Ejercicio: hornear un pastel

Analice la receta como un conjunto de objetos:

1

Objetos

¿Qué objetos participan al hornear un pastel?

2

Relaciones

¿Cómo se relacionan entre sí esos objetos?

3

Atributos

¿Qué atributos tiene cada uno? ¿Qué **tipo** de huevo?

4

Comportamiento

¿Qué puede hacer cada uno? El huevo solo puede romperse.

Usted ya sabe hacerlo: en la vida real no lo pensamos de forma explícita, pero todo objeto tiene **atributos** y **comportamiento**.

Definición de objeto

1

Estructura de datos

Agrupa **atributos** (datos) y **métodos** (funciones) que operan sobre esos datos.

2

Instancia de clase

La clase define la estructura (**atributos**) y el comportamiento (**métodos**).

3

Interfaz del objeto

Sus métodos y atributos **públicos**, accesibles desde otros objetos.

La mayoría de los lenguajes orientados a objetos requieren **definir una clase** para crear objetos.

// objetos y clases

Definición de clase: Televisor

Clase: plantilla de los objetos

```
class Televisor
{
    // Atributos
    public string marca;
    public int pulgadas;
    public bool encendido;

    // Métodos
    public void Encender() { encendido = true; }
    public void Apagar() { encendido = false; }
}
```

Objetos: instancias de la clase

```
Televisor samsung = new Televisor();
samsung.marca = "Samsung";
samsung.pulgadas = 55;

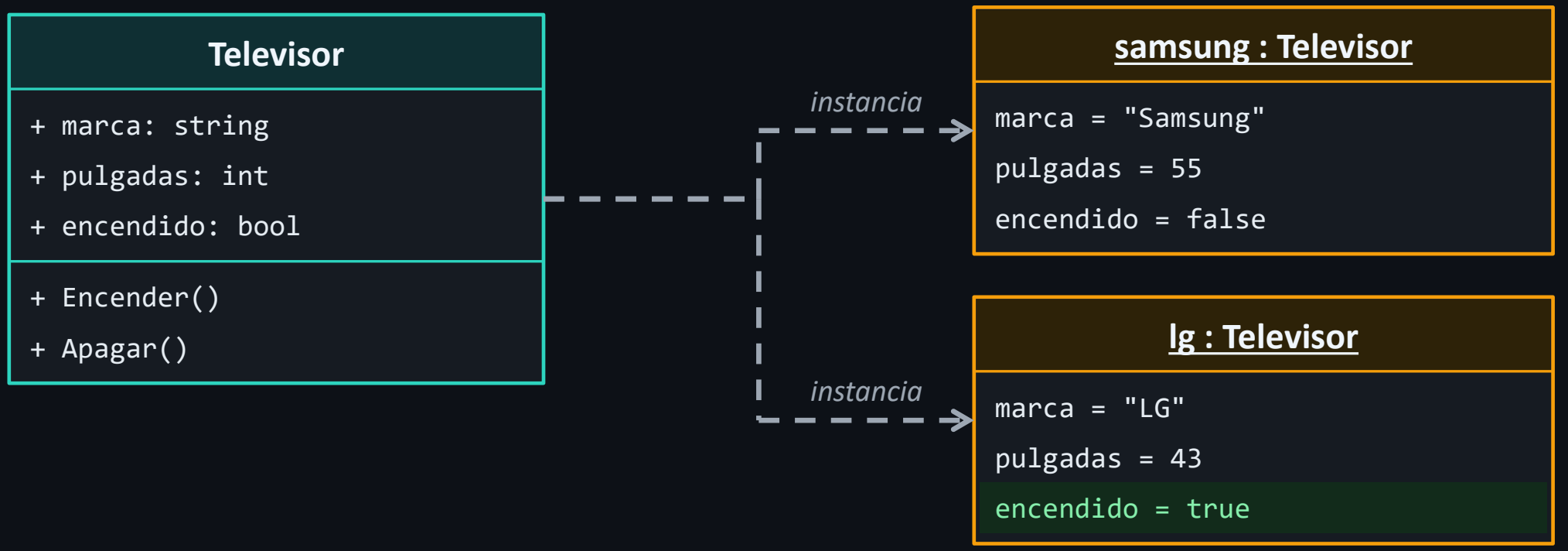
Televisor lg = new Televisor();
lg.marca = "LG";
lg.pulgadas = 43;
lg.Encender();

Console.WriteLine(samsung.encendido);
// Imprime false
```

Clase vs. objeto: la clase es un **plano o molde**; con new se **crean** (instancian) objetos a partir de ella.

Recuerde: definir clases es **extender el lenguaje** con nuevos tipos de datos compuestos.

De la clase a los objetos



Cada objeto es un **bloque de memoria independiente**: `lg.Encender()` no cambia el estado de `samsung`.

// principios de poo

Los cuatro principios de la POO

A

Abstracción

Modelar el problema con objetos de **alto nivel** que ocultan su implementación.

E

Encapsulamiento

Asociar código y datos; **exponer solo la interfaz** necesaria.

H

Herencia

Jerarquías de clases para **reutilizar código** (relación «es un/a»).

P

Polimorfismo

Un objeto puede comportarse de **muchas formas** distintas.

Son los elementos que **distinguen** la programación orientada a objetos de otros paradigmas.

// abstracción

Abstracción

1

Modelar el problema

Con objetos de **alto nivel** que **ocultan** los detalles de su implementación.

2

Clases y objetos

Representan **entidades del mundo real**: `Persona` con nombre, edad y métodos.

3

Comportamiento

Pensamos en **comportamiento y atributos**, no en variables y procedimientos.

Ojo: la palabra reservada `abstract` de algunos lenguajes **no** tiene relación con la abstracción como principio de POO.

// abstracción

Abstracción: quedarse con lo relevante

1

La realidad

Una casa real: paredes, ventanas, **tuberías**, **cableado**…

2

Se omite

Los detalles que **no son relevantes** para el propósito del dibujo.

3

La abstracción

Un dibujo de la casa: solo lo **relevante**, más fácil de manejar.

En POO, `Televisor` modela `marca`, `pulgadas` y `encendido`, no sus circuitos. La abstracción decide **qué modelar**; el encapsulamiento, **qué ocultar y qué exponer**.

// abstracción

Piezas de LEGO vs. plasticina

Piezas de LEGO

- Tienen una **interfaz** específica para conectarse
- No necesitamos saber cómo están construidas por dentro
- Clara separación entre interfaz e implementación

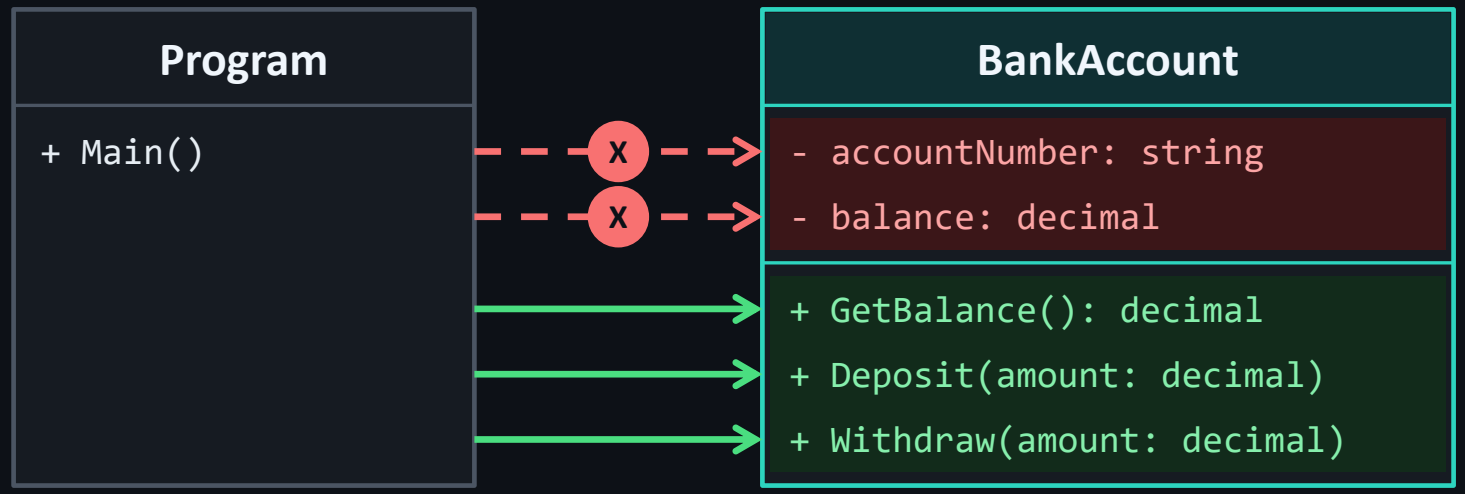
vs

Plasticina

- **No hay** una clara separación
- Todo forma parte de **un todo**
- No hay interfaz que oculte los detalles

La abstracción hace que los objetos se comporten como **piezas de LEGO**.

Encapsulamiento: la clase BankAccount



Interfaz vs. detalle

- `private`: solo accesible desde la misma clase
- `public`: la interfaz que se expone
- `account.balance = …` no compila (error **CS0122**)

Se **ocultan** los detalles de implementación y se expone **solo la interfaz** necesaria.

// encapsulamiento

BankAccount: ocultar datos, exponer métodos

Clase BankAccount (recortada)

```
public class BankAccount
{
    private string accountNumber;
    private decimal balance;

    public decimal GetBalance()
    {
        return balance;  // una copia
    }
    // + Deposit(...) y Withdraw(...)
}
```

Código externo

```
var account = new BankAccount("1234567890");
account.Deposit(1000);
account.Withdraw(500);
decimal balance = account.GetBalance();

account.balance = 1000000;  // error
```

Acceso controlado

`balance` solo cambia con `Deposit()` y `Withdraw()`.

Error CS0122: `'BankAccount.balance' is inaccessible due to its protection level`

// encapsulamiento

Modificadores de acceso (C#)

Modificador	Accesible desde	Símbolo UML
public	Cualquier parte del código	+
private	Solo la misma clase	-
protected	La misma clase y sus clases derivadas	#

El programador decide **qué se expone** y qué se oculta, como en un microondas: **botones** públicos, **interior** privado.

// encapsulamiento

Acceso controlado con get y set

```
public class Persona
{
    private string nombre;

    public string GetNombre()
    {
        return nombre;
    }

    public void SetNombre(string nombre)
    {
        this.nombre = nombre;
    }
}
```

Atributo privado

`nombre` solo es accesible desde la misma clase.

Métodos públicos

`GetNombre()` y `SetNombre()` permiten leer y modificar `nombre` de forma **controlada**.

Herencia: Animal y Perro



Superclase y subclase

- **Superclase** o clase base (`Animal`): comportamiento general
- **Subclase** o clase derivada (`Perro`): comportamiento especializado
- Cada clase tiene **máximo una** clase padre
- Relación «es un/a»: un perro **es un** animal

Se hereda para **reutilizar código**: `Perro` obtiene `Nombre`, `Edad` y `Comer()` de `Animal`.

// herencia

Superclase: Animal

Definición de la clase base

```
public class Animal
{
    public string Nombre { get; set; }
    public int Edad { get; set; }

    public void Comer()
    {
        Console.WriteLine("El animal está comiendo");
    }
}
```

Comportamiento general

`Nombre`, `Edad` y `Comer()` sirven para cualquier animal.

Propiedades de C#

`{ get; set; }`: leer y modificar el valor.

La superclase provee el comportamiento **general**; las subclases lo **especializan**.

// herencia

Subclase: Perro hereda de Animal

La subclase agrega comportamiento

```
public class Perro : Animal
{
    public void Ladrar()
    {
        Console.WriteLine("El perro está ladrando");
    }
}
```

Uso

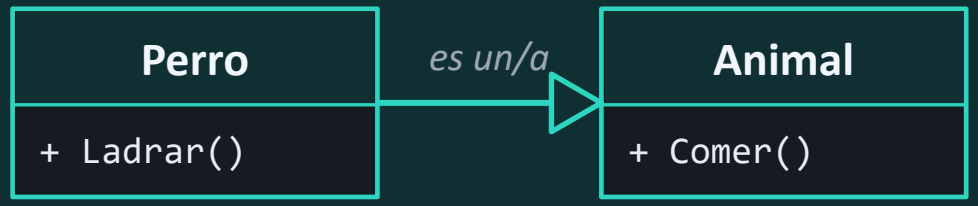
```
Perro perro = new Perro();
perro.Nombre = "Firulais";
perro.Edad = 3;
perro.Comer();   // heredado
perro.Ladrar();  // propio
```

`Perro : Animal` significa «**Perro hereda de Animal**»: no repite `Nombre`, `Edad` ni `Comer()`.

Herencia vs. composición

Herencia: «es un/a»

Una clase **se define a partir de otra**.

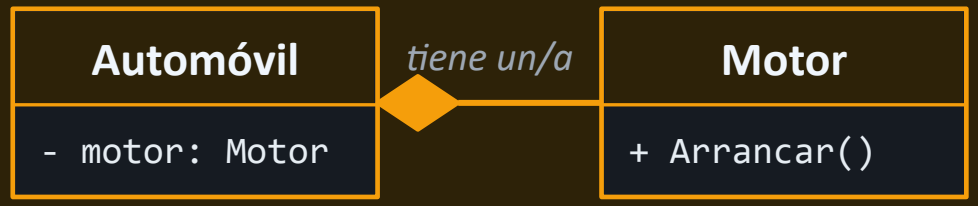



```
public class Perro : Animal
{
    // hereda Nombre, Edad y Comer()
}
```

vs

Composición: «tiene un/a»

Un objeto **contiene** a otro como atributo.




```
public class Automóvil
{
    private Motor motor = new Motor();
}
```

Antes de heredar, pregunte: ¿la relación es «**es un/a**» o «**tiene un/a**»? Un `Automóvil` no es un `Motor`: lo tiene.

// polimorfismo

Polimorfismo: «muchas formas»

En tiempo de ejecución

Una clase hija **anula** (override) un método `public` o `protected` de la clase padre.

- Se resuelve **al ejecutar** el programa
- Ejemplo: `Draw()` en `Shape` y `Circle`

vs

En tiempo de compilación

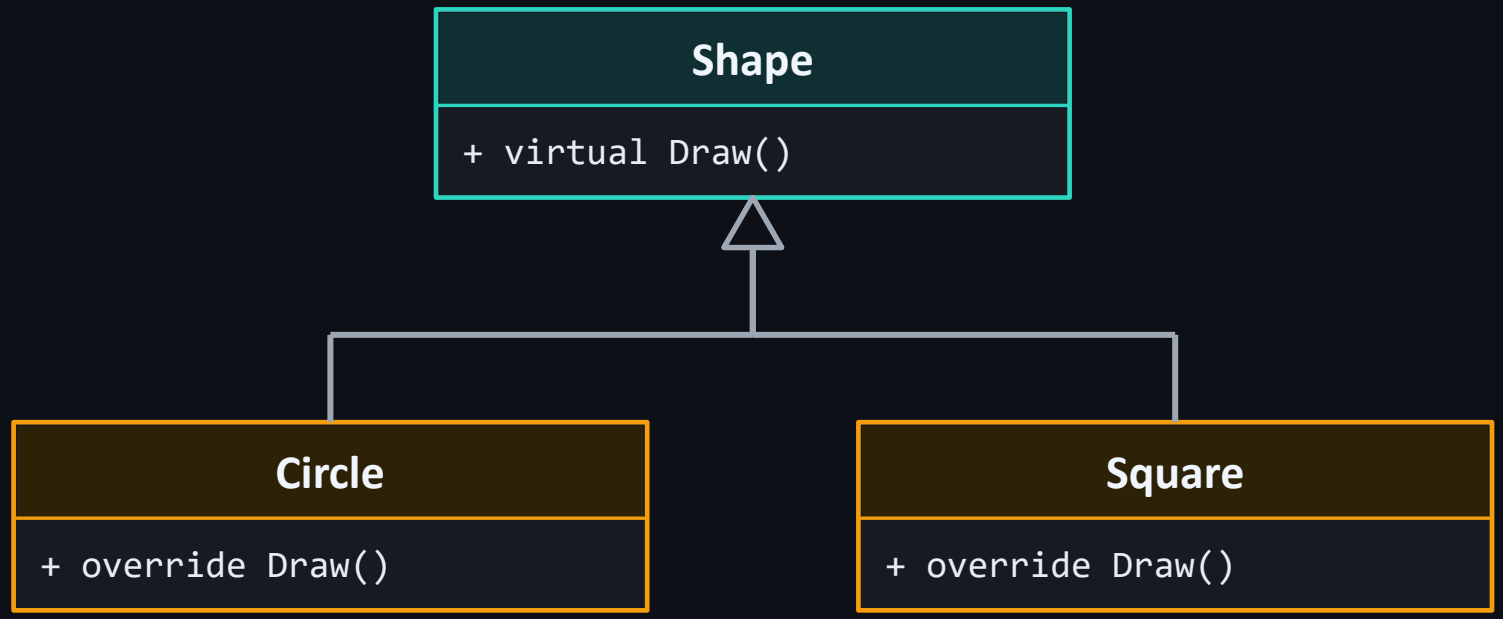
Sobrecarga (overload): varios métodos con el **mismo nombre** y **distintos parámetros**.

- Se resuelve **al compilar** el programa
- Ejemplo: `foo()` y `foo(int x)`

Polimorfismo: la capacidad de un objeto de **comportarse de diferentes maneras**.

// polimorfismo

Run-time: una jerarquía de figuras



Cómo se logra

- `virtual`: el padre permite que lo anulen
- `override`: la hija redefine `Draw()`
- Una variable `Shape` puede referirse a un `Circle` o a un `Square`

Cada subclase **anula** `Draw()` con su propio comportamiento.

// polimorfismo

Run-time: sobrescribir con override

Clase padre

```
public class Shape
{
    public virtual void Draw()
    {
        Console.WriteLine("Drawing a shape");
    }
}
```

Clase hija

```
public class Circle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing a circle");
    }
}
```

virtual: la clase padre **permite** que `Draw()` sea anulado.

override: la clase hija **redefine** `Draw()`.

Square: es igual; imprime «Drawing a square».

// polimorfismo

¿Qué método se ejecuta?

Código cliente

```
public static void Main(string[] args)
{
    Shape shape1 = new Circle();
    Shape shape2 = new Square();

    shape1.Draw(); // Drawing a circle
    shape2.Draw(); // Drawing a square
}
```

Se resuelve al llamarlo

El método correcto se determina **al llamarlo**.

Tipo declarado

`shape1` se declara como `Shape`.

Clase real del objeto

En ejecución, `shape1` es un `Circle`.

Búsqueda del método

Se busca `Draw()` en `Circle`; si no está, se sube por la jerarquía.

// polimorfismo

Polimorfismo en acción: una lista de figuras

Código cliente

```
Shape[] figuras =
    { new Circle(), new Square(), new Circle() };

foreach (Shape figura in figuras)
{
    figura.Draw();
}
```

Salida

```
Drawing a circle
Drawing a square
Drawing a circle
```

Sin condicionales

El ciclo no pregunta si el objeto es un `Circle` o un `Square`.

El código cliente trata todos los objetos como `Shape`; cada uno ejecuta **su** `Draw()`.

Beneficio: agregar otra subclase de `Shape` **no obliga a cambiar** el ciclo.

// polimorfismo

Tratar a la hija como si fuera la padre

Visible en la clase padre

Todo atributo o método **visible** de `Animal` se accede a través de la hija.

```
Animal a = new Perro();
a.Comer();  // OK
```

vs

Específico de la hija

Lo que es **propio** de `Perro` no es accesible a través de la clase padre.

```
Animal a = new Perro();
a.Ladrar();  // error
```

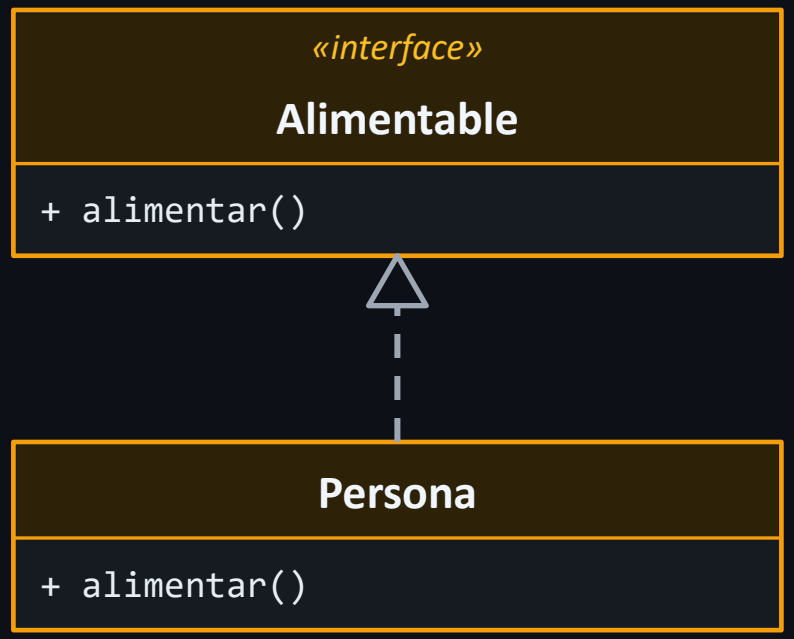
El polimorfismo permite **tratar a la clase hija como si fuera la padre**.

Clases abstractas e interfaces

Clase abstracta



Interfaz



La clase abstracta da polimorfismo **dentro de una jerarquía**; la interfaz, **sin formar parte de una jerarquía**.

// polimorfismo

Clase abstracta en C#

Clase abstracta

```
public abstract class Animal
{
    public void Respirar()
    {
        Console.Write("Respirando");
    }

    // sin definición: fuerza a las
    // hijas a implementarlo
    public abstract void PorDefinir();
}
```

Clase hija

```
public class Perro : Animal
{
    // si no lo implementa, el
    // compilador arroja un error
    public override void PorDefinir()
    {
        Console.Write("Foo...");
    }
}
```

Un método `abstract` **no tiene cuerpo**: obliga a cada hija a implementarlo.

Instanciación: una clase `abstract` **no se puede instanciar**: `new Animal()` no compila; `new Perro()` sí.

// polimorfismo

Interfaz en C#

Interfaz

```
public interface Alimentable
{
    void alimentar();
}
```

Clase que la implementa

```
public class Persona : Alimentable
{
    // Implementa el método de la interfaz.
    // Si no lo hace, el compilador arroja un error.
    public void alimentar()
    {
        Console.Write("Comiendo");
    }
}
```

Sin jerarquía

`Persona` no hereda de nadie y aun así es `Alimentable`.

Una **interfaz** da polimorfismo **sin formar parte de una jerarquía** de clases.

Buena práctica: declare `Alimentable a`, no `Persona a`: menos **acoplamiento** con la clase concreta.

// polimorfismo

Compile-time: sobrecarga de métodos

Mismo nombre, distintos parámetros

```
public class TestClass
{
    public void foo() { }

    public void foo(int x) { }

    public void foo(string s, int x) { }
}
```

Uso

```
var t = new TestClass();
t.foo();          // foo()
t.foo(7);         // foo(int x)
t.foo("a", 7);    // foo(string s, int x)
```

Sobrecarga (overload): el compilador elige la versión según los **argumentos** de la llamada.

// práctica

Ahora usted

Aplique lo aprendido a cada caso:

A

¿Es un/a?

Dadas `Vehículo` y `Automóvil`, ¿cuál es la superclase? Escriba la frase «es un/a».

B

Visibilidad

En `BankAccount`, ¿qué modificador usaría para `balance` y cuál para `Deposit()`?

C

¿Override u overload?

`Sumar(int a)` y `Sumar(int a, int b)` en una clase: ¿override u overload?

// cierre

Resumen

- **Tipos de datos:** simples o compuestos; de valor o de referencia. Los objetos son compuestos y de referencia.
- **TDA:** una interfaz sin implementación. TDA + implementación = **clase**.
- **Objeto y clase:** la POO modela el mundo con objetos (atributos y comportamiento); la clase es su plantilla.
- **Abstracción y encapsulamiento:** modelar solo lo relevante, ocultar los detalles y exponer la interfaz (`public` / `private`).
- **Herencia y polimorfismo:** «es un/a» (la composición es «tiene un/a»); `override` en ejecución y sobrecarga en compilación.

Referencias: Paradigma de programación (es.wikipedia.org/wiki/Paradigma_de_programación) · Microsoft Learn: tipos de valor, modificadores de acceso y propiedades en C# (learn.microsoft.com).